OTH Amberg-Weiden

Symbolic AI and Rule-based Agents

Foundations

Prof. Dr. Tatyana Ivanovska

Amberg, 04.05.2026

Overview

OTH Amberg-Weiden

Paradigms

OTH Amberg-Weiden

Two **fundamentally different** paradigms:

1. Agent-based modeling (ABM): simulation of many interacting agents to study the system dynamics, emergent behavior from simple interactions (e.g. Mesa, AgentPy)
2. Agent orchestration / LLM-based workflows (could be represented as graph-based control flow e.g. LangGraph)

Agent

OTH Amberg-Weiden

Agent is an entity that **perceives, reasons, and acts in an environment over time**

Dimension	Rule-based	ABM	LangGraph
Goal	decision logic	emergence	orchestration
Scale	few	many	few
Control	rules	decentralized	graph
Time	steps	simulation	execution

Phase 1 - Deterministic rule-based agent

OTH Amberg-Weiden

- Example:
  - vacuum world / grid world
- Goal:
  - internal logic = rules
  - no learning, no emergence

Phase 2 - From single agent to multi-agent system

OTH Amberg-Weiden

- "What happens if we have many such agents interacting?"
- ABM:
  - decentralized decision-making
  - local rules → global patterns
  - Frameworks asMesa or AgentPy

Important

Same rule-based logic, different scale → emergent behavior

Question

Create an example for a multi-agent system for ABM

Where ABM not that well suitable for?

OTH Amberg-Weiden

We want an agent to:

- Plan a multi-step task
- Call external tools
- Maintain structured reasoning steps

We are interested in

How do we control reasoning flow, not population dynamics

LangGraph

OTH Amberg-Weiden

- LangGraph as control-flow graph
- LangGraph is an MIT-licensed open-source library and is free to use
- `https://www.langchain.com/langgraph`
- LangGraph is a programmatic reasoning graph, not a simulation!

Graphs are everywhere!

OTH Amberg-Weiden

- Rule-based agent
  - inference graph (rules)
- ABM
  - interaction graph (agents)
- LangGraph (orchestration)
  - execution graph (workflow)

Graphs!

Same mathematical object, different semantics

What happens if we insert LLM-Agents into ABM?

OTH Amberg-Weiden

- Classical ABM
  - rules are explicit
  - deterministic or stochastic
  - interpretable
- LLM-based ABM
  - rules become implicit
  - behavior is context-dependent
  - memory
  - language reasoning
  - non-determinism

When does it make sense?

OTH Amberg-Weiden

LLMs are useful in ABM when agents need:

- Rich cognition
  - negotiation
  - planning
  - belief updates
  - reasoning
- Natural language interaction
  - dialogue between agents
  - human-agent interaction
- Complex decision policies replacing handcrafted rules

Typical domains:

- social simulations
- economic behavior
- crowd dynamics with communication

When it does NOT make sense?

OTH Amberg-Weiden

LLMs are overkill when:

- rules are simple (e.g., Schelling segregation, cellular automata)
- large-scale simulation ($10^4 - -10^6$ agents): computationally infeasible
- strict reproducibility is required

Key limitation

OTH Amberg-Weiden

- ABM relies on:
  - controlled experiments
  - reproducibility
  - parameter sensitivity analysis
- LLMs introduce:
  - stochasticity
  - hidden internal state
  - prompt sensitivity

Conceptual Example

OTH Amberg-Weiden

```python
class Agent:
    def step(self, observation):
        action = self.policy(observation)
        return action

# classical
def rule_policy(obs):
    if obs["dirty"]:
        return "clean"
    return "move"

# LLM-based
def llm_policy(obs):
    prompt = f"State: {obs}. What should the agent do?"
    return call_llm(prompt)
```

What is agent orchestration?

Agent orchestration = explicit control of how one or more agents, tools, and reasoning steps are

- executed,
- coordinated,
- and state-managed to complete a task.

Orchestration = (Graph, State, Transition Rules)

Element	Meaning
Graph	workflow topology
Nodes	actions (LLM, tool, function)
Edges	conditional transitions
State	persistent memory

When does orchestration make sense?

OTH Amberg-Weiden

Multi-step reasoning with control

- Example:
  plan → retrieve → reason → verify → revise
- Required:
  - loops
  - conditional branching
  - state tracking

Important

This is not ABM and not a simple rule-based agent

When does orchestration make sense?

OTH Amberg-Weiden

Tool-augmented agents

- Example:
  - call database
  - call API
  - run code
- You must control:
  - when tools are invoked
  - how results are reused

Important

This is not ABM and not a simple rule-based agent

When does orchestration make sense?

OTH Amberg-Weiden

Reliability / production constraints

- LLMs alone are:
  - stochastic
  - fragile
- Orchestration adds:
  - retries
  - validation steps
  - fallback paths

Important

This is not ABM and not a simple rule-based agent

When does orchestration make sense?

OTH Amberg-Weiden

Multi-agent cooperation (small scale)

- Example:
  - planner agent
  - executor agent
  - critic agent
- This is coordination, not ABM-style emergence!

Important

This is not ABM and not a simple rule-based agent

ABM and Orchestration

OTH Amberg-Weiden

Aspect	ABM (Mesa / AgentPy)	LangGraph
Goal	simulate systems	solve tasks
Time	evolving system	execution steps
Agents	many (100–10^6)	few (1–10)
Behavior	emergent	controlled
Control	decentralized	centralized graph

Table: Comparison of Agent-Based Models and LangGraph

An (advanced) vacuum cleaner world

OTH Amberg-Weiden

- Simple formulation: if dirty -> clean (**NO Orchestration is needed!**)
- Advanced formulation:
  - vacuum cleaner world, where the agent needs not just clean the waste, but also to analyze the type of the waste and select the appropriate processing strategy.
  - **Orchestration:** perceive -> analyze waste -> choose strategy -> clean -> verify -> repeat or stop

LangGraph

- models this as a stateful execution graph
- a StateGraph consists of nodes that read/write a shared state
- each node returns a partial state update
- Control flow is defined using normal edges or conditional edges

LangGraph

OTH Amberg-Weiden

- In this course, no extensive explanations on LLM Agents
- Documentation: `https://docs.langchain.com/oss/python/langgraph/graph-api`
- Here, only main concepts to introduce some basic understanding

Main LangGraph concepts

OTH Amberg-Weiden

The state is the shared memory of the workflow.

```python
class VacuumState(TypedDict):
    location: str
    waste_detected: bool
    waste_type: str
    action: str
    cleaned: bool
    attempts: int
```

Main LangGraph concepts

OTH Amberg-Weiden

A node is a function that receives the current state and returns updates.

```
def analyze_waste(state: VacuumState):
    ...
    return {"waste_type": "liquid"}
```

Main LangGraph concepts

OTH Amberg-Weiden

Edges define the order of execution.

```
graph.add_edge("perceive", "analyze")
graph.add_edge("analyze", "choose_action")
```

Main LangGraph concepts

OTH Amberg-Weiden

Conditional edges decide dynamically where to go next.

```
def route_after_verification(state):
    if state["cleaned"]:
        return "end"
    return "retry"
```

LangGraph supports conditional edges through routing functions that inspect the current state and return the next node.

Vacuum cleaner world: problem definition

OTH Amberg-Weiden

- The agent moves in a small environment
- Each cell may contain different waste types
- So the agent should not only detect that a cell is dirty. It must classify the waste first

Waste type	Cleaning strategy
dust	vacuum
paper	pick up
liquid	mop
glass	avoid and report

Table: Waste types and corresponding cleaning strategies

Graph

OTH Amberg-Weiden

```
START
  ↓
perceive_environment
  ↓
analyze_waste
  ↓
choose_action
  ↓
perform_action
  ↓
verify_cleaning
  ↓
conditional decision:
      if cleaned       -> END
      if needs retry   -> choose_action
      if unsafe waste  -> report_problem -> END
```

This is orchestration: we explicitly control the reasoning and action steps.

Code

OTH Amberg-Weiden

```python
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END


# ============================================================
# 1. Define the shared state
# ============================================================

class VacuumState(TypedDict):
    location: str
    sensor_input: str
    waste_detected: bool
    waste_type: str
    action: str
    cleaned: bool
    unsafe: bool
    attempts: int
    log: list[str]
```

Code

OTH Amberg-Weiden

```python
# ============================================================
# 2. Define nodes
# ============================================================

def perceive_environment(state: VacuumState):
    """
    Simulates perception.
    In a real robot, this could read camera/sensor data.
    """
    sensor_input = state["sensor_input"]

    waste_detected = sensor_input != "clean"

    return {
        "waste_detected": waste_detected,
        "log": state["log"] + [f"Perceived sensor input: {sensor_input}"]
    }
```

Code

OTH Amberg-Weiden

```python
def analyze_waste(state: VacuumState):
    """
    Classifies the waste type.
    In a real system, this could be an image classifier or an LLM/VLM call.
    """
    if not state["waste_detected"]:
        waste_type = "none"
    else:
        sensor_input = state["sensor_input"].lower()

        if sensor_input in ["dust", "sand", "crumbs"]:
            waste_type = "dry_small"
        elif sensor_input in ["paper", "plastic"]:
            waste_type = "solid_object"
        elif sensor_input in ["water", "juice", "coffee"]:
            waste_type = "liquid"
        elif sensor_input in ["glass", "needle"]:
            waste_type = "hazardous"
        else:
            waste_type = "unknown"

    return {
        "waste_type": waste_type,
        "log": state["log"] + [f"Analyzed waste type: {waste_type}"]
    }
```

Code

OTH Amberg-Weiden

```python
def choose_action(state: VacuumState):
    """
    Chooses an action based on classified waste type.
    """
    waste_type = state["waste_type"]

    if waste_type == "none":
        action = "do_nothing"
        unsafe = False
    elif waste_type == "dry_small":
        action = "vacuum"
        unsafe = False
    elif waste_type == "solid_object":
        action = "pick_up"
        unsafe = False
    elif waste_type == "liquid":
        action = "mop"
        unsafe = False
    elif waste_type == "hazardous":
        action = "avoid_and_report"
        unsafe = True
    else:
        action = "request_human_help"
        unsafe = True
```

Code

```python
def perform_action(state: VacuumState):
    """
    Executes the action.
    """
    action = state["action"]

    if action in ["vacuum", "pick_up", "mop", "do_nothing"]:
        cleaned = True
    else:
        cleaned = False

    return {
        "cleaned": cleaned,
        "attempts": state["attempts"] + 1,
        "log": state["log"] + [f"Performed action: {action}"]
    }


def verify_cleaning(state: VacuumState):
    """
    Verifies whether the cleaning was successful.
    """
    if state["cleaned"]:
        message = "Verification: area is clean."
    elif state["unsafe"]:
        message = "Verification: unsafe waste, cleaning stopped."
```

Code

OTH Amberg-Weiden

```python
def report_problem(state: VacuumState):
    """
    Handles unsafe or unknown waste.
    """
    return {
        "log": state["log"] + [
            f"Reported problem at {state['location']}: {state['waste_type']}"
        ]
    }


# ============================================================
```

Code

OTH Amberg-Weiden

```python
# ============================================================
# 3. Define routing logic
# ============================================================

def route_after_verification(state: VacuumState):
    """
    Decides where to go after verification.
    """
    if state["cleaned"]:
        return "end"

    if state["unsafe"]:
        return "report"

    if state["attempts"] >= 2:
        return "report"

    return "retry"
```

Code

OTH Amberg-Weiden

```
# ============================================================
# 4. Build the graph
# ============================================================

graph = StateGraph(VacuumState)

graph.add_node("perceive_environment", perceive_environment)
graph.add_node("analyze_waste", analyze_waste)
graph.add_node("choose_action", choose_action)
graph.add_node("perform_action", perform_action)
graph.add_node("verify_cleaning", verify_cleaning)
graph.add_node("report_problem", report_problem)

graph.add_edge(START, "perceive_environment")
graph.add_edge("perceive_environment", "analyze_waste")
graph.add_edge("analyze_waste", "choose_action")
graph.add_edge("choose_action", "perform_action")
graph.add_edge("perform_action", "verify_cleaning")

graph.add_conditional_edges(
    "verify_cleaning",
    route_after_verification,
    {
        "end": END,
        "retry": "choose_action",
        "report": "report_problem",
    }
)

graph.add_edge("report_problem", END)

app = graph.compile()
```

Code

OTH Amberg-Weiden

```
# ============================================================
# 5. Run example cases
# ============================================================

def run_case(sensor_input: str):
    initial_state: VacuumState = {
        "location": "cell_A",
        "sensor_input": sensor_input,
        "waste_detected": False,
        "waste_type": "",
        "action": "",
        "cleaned": False,
        "unsafe": False,
        "attempts": 0,
        "log": [],
    }

    result = app.invoke(initial_state)

    print(f"\n=== Case: {sensor_input} ===")
    for item in result["log"]:
        print("-", item)
```

Controlled decision processes

OTH Amberg-Weiden

Concept	In this example
State	current knowledge of the robot
Node	one reasoning/action step
Edge	fixed execution order
Conditional edge	dynamic decision after verification
END	stopping condition

Table: Concepts in a graph-based reasoning workflow

Expected execution

OTH Amberg-Weiden

```
=== Case: coffee ===
- Perceived sensor input: coffee
- Analyzed waste type: liquid
- Chosen action: mop
- Performed action: mop
- Verification: area is clean.
```

How to extend? Additional tasks:

OTH Amberg-Weiden

- Add battery level
- Add multiple rooms
- Add 'move to next dirty cell'
- Add real image classifier for waste analysis
- Add an LLM/VLM node that classifies waste from text or image
- Extend the graph so the robot must decide between cleaning, moving, recharging, or reporting. The solution must contain at least one conditional edge and at least one retry loop

Overview

OTH Amberg-Weiden

01 | Agentic Workflow

02 | **Project**

OTH Amberg-Weiden

Topic: Waste in the city

The goal is to study how different types of agents and infrastructure influence the spatial and temporal distribution of waste in the city.

- The city is modeled as a grid world consisting of:
  - streets / paths where agents can move
  - walls / buildings that block movement
  - public areas where waste may accumulate
  - fixed waste infrastructure, such as bins and containers

Agent types

Your model should include at least the following entities:

1. Local humans
   - move according to regular daily patterns
   - generate small amounts of waste with a given probability
   - may use nearby bins if available
2. Tourists
   - move less predictably
   - prefer central or attractive areas
   - may generate more waste in crowded areas
3. Cleaning service
   - move through streets
   - detect and collect waste from public spaces and bring to the disposal areas, where waster is processed/terminated
   - may follow simple rule-based strategies, e.g. nearest waste, random patrol, or fixed route

Agent types

4. Dust bins and dust containers
   - fixed infrastructure
   - have limited capacity
   - receive waste from humans and robots
   - may overflow if not emptied
5. Dust transporters
   - collect waste from full bins or containers
   - transport it to a disposal point outside or at the edge of the city
   - follow street paths and cannot pass through buildings

Main research questions

OTH Amberg-Weiden

How do city structure, movement patterns, bin placement, cleaning strategies, and transporter frequency affect waste accumulation in a city?

ABM task

The main interest is not one agent's decision process, but the **emergent city-level waste distribution caused by many interacting agents over time**.

Experiments

OTH Amberg-Weiden

1. Generate a city with a given structure
2. Run multiple scenarios:
   - Many streets/Many buildings
   - Few bins vs. many bins
   - Random robot patrol vs. nearest-waste strategy
   - Low tourist density vs. high tourist density
   - Rare transporter visits vs. frequent transporter visits
3. Measure and visualize statistics over simulation steps
   - total waste on streets
   - number of overflowing bins
   - average waste per district / city region
   - robot cleaning efficiency
   - transporter workload
4. Implement your simulation in Mesa or AgentPy
5. Use graph search algorithms where it makes sense

Experiments

OTH Amberg-Weiden

Creative Extension Requirement (Mandatory)

In addition to the core implementation, each group must

- design and
- implement **an original extension that meaningfully enhances the model**

Repetition

If two or groups will have **identical** 'creative extensions', the evaluation will be reduced for these groups (as no creative solution is implemented!)

Project DATES!

OTH Amberg-Weiden

- **11.05. - 01.06.2026** are reserved for your project implementations (NO LECTURES)
- Please work in **2-people-teams**
- On **01.06.2026** submit the **Code, Documentation, Presentation** to Moodle
- Presentation: 10 Minutes + 5-10 Minutes for Questions (per person in a team)
- You are allowed to use Chatbots, but if your input is copy-paste from ChatGPT or any other source, please do not expect to pass
- **08.06** (and if needed **15.06**) Project presentations with understanding questions. You must be able to explain the parts of your code and demonstrate the understanding of what you implemented, i.e. project defense

Questions

OTH Amberg-Weiden

Thank you for your attention! I am open to your questions!

Contact

OTH Amberg-Weiden

Prof. Dr. Tatyana Ivanovska

Prof. for Visual AI

Ostbayerische Technische Hochschule (OTH) Amberg-Weiden
Kaiser-Wilhelm-Ring 23 | 92224 Amberg

Tel.: +49 (9621) 482-3652
Fax: +49 (9621) 482-4642
t.ivanovska@oth-aw.de

OTH Amberg-Weiden

Thank you!

Prof. Dr. Tatyana Ivanovska
Ostbayerische Technische Hochschule (OTH) Amberg-Weiden
Kaiser-Wilhelm-Ring 23 | 92224 Amberg

Tel.: +49 (9621) 482-3652
Fax: +49 (9621) 482-4642
t.ivanovska@oth-aw.de